

Suvenýry

Erik si kupuje suvenýry v zahraničním obchůdku. Existuje celkem N **typů** suvenýrů. V obchůdku je nekonečně mnoho suvenýrů každého typu.

Každý typ suvenýru má pevně danou cenu. Konkrétně suvenýr typu i ($0 \leq i < N$) stojí $P[i]$ BOBů, kde $P[i]$ je kladné celé číslo.

Erik ví, že typy suvenýrů jsou seřazeny sestupně dle ceny a že ceny suvenýrů jsou různé. Přesněji $P[0] > P[1] > \dots > P[N-1] > 0$. Navíc se mu podařilo zjistit hodnotu $P[0]$. Žádné další informace o cenách suvenýrů Erik bohužel nemá.

Pro nákup suvenýrů Erik provede několik transakcí s prodejcem.

Každá transakce sestává z následujících kroků:

1. Erik předá nějaké (kladné) množství BOBů prodejci.
2. Prodejce tyto BOBy umístí na stůl ve skladišti, kam Erik nedohlédne.
3. Prodejce si postupně zauvažuje o každém z typů suvenýru v pořadí $0, 1, \dots, N-1$. Každý typ zauvažuje **právě jednou** v rámci transakce.
 - Při uvažování suvenýru typu i , je-li aktuální počet BOBů na stole alespoň $P[i]$:
 - prodejce oddělá $P[i]$ BOBů ze stolu a nahradí je jedním suvenýrem typu i .
4. Nakonec prodejce vrátí Erikovi všechny zbývající BOBy a předá mu suvenýry ze stolu.

Poznamenejme, že před začátkem transakce je stůl vždy prázdný.

Vaším úkolem je pomoci Erikovi provést nějaký počet transakcí tak, aby:

- v každé transakci nakoupil **alespoň jeden** suvenýr a
- dohromady nakoupil **právě** i suvenýrů typu i pro každé i takové, že $0 \leq i < N$.
Podotkněme, že by si tudíž Erik nikdy neměl zakoupit suvenýr typu 0.

Erik nemusí minimalizovat počet transakcí a má přístup k bankovní tiskárně, totiž neomezenému zdroji BOBů.

Implementační detaily

Měli byste naimplementovat následující funkci:

```
void buy_souvenirs(int N, long long P0)
```

- N : počet typů suvenýrů.
- $P0$: hodnota $P[0]$.
- Tato funkce je zavolána právě jednou pro každý vstup.

Funkce výše může volat následující funkci pro přikázání Erikovi provést transakci:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : počet BOBů předaných prodejci Erikem.
- Funkce vrací pár. Prvním prvkem páru je pole L obsahující nakoupené typy suvenýrů (ve vzestupném pořadí). Druhým prvkem je celé číslo R , počet BOBů vrácených Erikovi po transakci.
- Je vyžadováno, aby $P[0] > M \geq P[N - 1]$. Podmínka $P[0] > M$ zaručuje, že si Erik nekoupí suvenýr typu 0. Podmínka $M \geq P[N - 1]$ zase zaručuje, že si Erik koupí alespoň jeden suvenýr. Pokud tyto podmínky nejsou splněny, vaše řešení dostane verdikt `Output isn't correct: Invalid argument` (Výstup není správně: Neplatný parametr). Upozorníme, že narozdíl od $P[0]$ není hodnota $P[N - 1]$ dostupná na vstupu.
- Funkce může být zavolána nanejvýš 5000krát pro každý vstup.

Chování graderu **není adaptivní**. To znamená, že posloupnost cen P je pevně daná před zavoláním `buy_souvenirs`.

Omezení

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ pro každé i takové, že $0 \leq i < N$.
- $P[i] > P[i + 1]$ pro každé i takové, že $0 \leq i < N - 1$.

Podúlohy

Podúloha	Počet bodů	Další omezení
1	4	$N = 2$
2	3	$P[i] = N - i$ pro každé i takové, že $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ pro každé i takové, že $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ pro každé i takové, že $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ pro každé i takové, že $0 \leq i < N - 1$.
6	33	Žádná další omezení.

Příklad

Uvažujme následující volání:

```
buy_souvenirs(3, 4)
```

Máme $N = 3$ typů suvenýrů a $P[0] = 4$. Všimněme si, že existují pouze tři možné posloupnosti cen P : $[4, 3, 2]$, $[4, 3, 1]$ a $[4, 2, 1]$.

Nechť `buy_souvenirs` zavolá `transaction(2)`. Uvažujme, že funkce vrátí $([2], 1)$, tedy že Erik dostal jeden suvenýr typu 2 a že mu prodejce vrátil 1 BOB. Nahlédněme, že nám to umožní odvodit $P = [4, 3, 1]$, jelikož:

- Pro $P = [4, 3, 2]$ by `transaction(2)` vrátila $([2], 0)$.
- Pro $P = [4, 2, 1]$ by `transaction(2)` vrátila $([1], 0)$.

Následně může `buy_souvenirs` zavolat `transaction(3)`, která vrátí $([1], 0)$, totiž že Erik koupil jeden suvenýr typu 1 a nedostal zpátky žádné BOBy. Doposud zakoupil jeden suvenýr typu 1 a jeden suvenýr typu 2.

Na konec může `buy_souvenirs` zavolat `transaction(1)`, která vrátí $([2], 0)$, totiž že Erik zakoupil jeden suvenýr typu 2. Poznamenejme, že by taktéž šlo použít `transaction(2)`. V tento okamžik má Erik celkem jeden suvenýr typu 1 a dva suvenýry typu 2, jak bylo požadováno.

Ukázkový grader

Formát vstupu:

```
N
P[0] P[1] ... P[N-1]
```

Formát výstupu:

```
Q[0] Q[1] ... Q[N-1]
```

Zde $Q[i]$ představuje celkový počet koupených suvenýrů typu i pro každé i takové, že $0 \leq i < N$.